

Introduction à pyGame

Table des matières

Installation.....	2
Pour Windows.....	2
Pour Linux.....	2
Création d'une fenêtre.....	2
Dessin à l'écran.....	3
Rectangle.....	3
Texte.....	4
Affichage des images.....	6
Gestion des évènements.....	8
Souris.....	9
Clavier.....	9
Associer un évènement à un objet graphique.....	10
Détection de collisions.....	12
Musique.....	17
Manipulation d'image.....	18
Références.....	19

Document écrit par Stéphane Gill

Mai 2023



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

pyGame est une bibliothèque libre multiplate-forme qui facilite le développement de jeux vidéo temps réel avec le langage de programmation Python.

Construite sur la bibliothèque SDL, elle permet de programmer la partie multimédia (graphismes, son et entrées au clavier, à la souris ou au joystick), sans se heurter aux difficultés des langages de bas niveaux comme le C et ses dérivés.

On peut aussi remarquer que pyGame n'est plus utilisée exclusivement pour des jeux vidéo, mais également pour des applications diverses nécessitant du graphisme.

Installation

Pour Windows

Pour Windows le plus simple pour installer pygame est d'utiliser pip. Pour cela, il va falloir également ouvrir un terminal (win + R et on tape cmd). Ensuite on installe pygame:

```
python -m pip install pygame
```

Pour Linux

À partir du terminal taper la commande suivante :

```
sudo pip install pygame
```

Création d'une fenêtre

Dans cette section, nous allons voir comment créer une fenêtre, et quelles possibilités nous sont offertes pour la personnaliser.

Exemple 1 : Dessiner un cercle bleu.

```
1 import pygame
2 pygame.init()
3
4 # Définir les couleurs (RGB)
5 BLUE = (0, 0, 255)
6 BLANC = (255, 255, 255)
7
8 # Définir la taille de la fenêtre
9 screen = pygame.display.set_mode([500, 500])
10
```

```

11 running = True
12
13 while running:
14     # Lorsque l'utilisateur click sur le X de la
15     # fenêtre quitter le programme
16     for event in pygame.event.get():
17         if event.type == pygame.QUIT:
18             running = False
19
20     # Définir l'arriere-plan
21     screen.fill(BLANC)
22
23     # Dessiner un cercle bleu au centre de la fenetre
24     pygame.draw.circle(screen, BLUE, (250, 250), 75, 2)
25
26     # Mettre à jour l'affichage
27     pygame.display.flip()
28
29 # Quitter correctement le programme
30 pygame.quit()
31

```

La méthode `set_mode()` permet de définir la taille de la fenêtre. Il est possible de spécifier le mode plein écran (*FULLSCREEN*).

La méthode `circle()` permet de dessiner un cercle. Sa syntaxe est la suivante :

```
pygame.draw.circle(screen, color, (x,y), radius, thickness)
```

où :

- `(x,y)` est la position du centre du cercle,
- `radius` est le rayon du cercle,
- `thickness` est l'épaisseur des lignes. Si une largeur de zéro est spécifiée, le cercle sera plein.

Dessin à l'écran

Rectangle

Exemple 2 : Dessiner un rectangle

```
1 import pygame
```

```

2  pygame.init()
3
4  BLEU = (0, 0, 255)
5  BLANC = (255, 255, 255)
6
7  screen = pygame.display.set_mode([500, 500])
8
9  running = True
10
11 while running:
12
13     for event in pygame.event.get():
14         if event.type == pygame.QUIT:
15             running = False
16
17     screen.fill(BLANC)
18
19     pygame.draw.rect(screen, BLEU, (0, 0, 100, 200), 2)
20
21     pygame.display.flip()
22
23 pygame.quit()

```

La méthode `rect()` permet de dessiner un rectangle. Sa syntaxe est la suivante :

```
rect(screen, color, (x,y,width,height), thickness)
```

où :

- X et y sont la position du coin supérieur gauche.
- Width et height sont la taille du rectangle.
- thickness est l'épaisseur du contour du cercle. Si une largeur de zéro est spécifiée, le cercle sera plein.

Texte

Exemple 3 : Afficher la liste des polices disponibles

```

1  import pygame
2
3  fonts = pygame.font.get_fonts()
4  print(len(fonts))
5  for f in fonts:
6      print(f)

```

Exemple 4 : Afficher la police par défaut

```
1 import pygame
2
3 sysfont = pygame.font.get_default_font()
4 print('system font :', sysfont)
```

Exemple 5 : Afficher du texte

```
1 import pygame
2 pygame.init()
3
4 BLEU = (0, 0, 255)
5 BLANC = (255, 255, 255)
6 ROUGE = (255, 0, 0)
7
8 screen = pygame.display.set_mode([500, 500])
9
10 # Charger la police de caractère
11 sysfont = pygame.font.get_default_font()
12 font = pygame.font.SysFont(None, 48)
13
14 running = True
15
16 while running:
17     for event in pygame.event.get():
18         if event.type == pygame.QUIT:
19             running = False
20
21     screen.fill(BLANC)
22
23     img = font.render("Bonjour!", True, ROUGE)
24     screen.blit(img, (20, 20))
25
26     pygame.display.flip()
27
28
29 pygame.quit()
```

Exemple 6 : Afficher un cadre autour d'un texte

```
30 ...
1     img = font.render("Bonjour!", True, ROUGE)
2     rect = img.get_rect()
3     pygame.draw.rect(img, BLEU, rect, 1)
4     screen.blit(img, (20, 60))
5     ...
```

Exemple 7 : Créer un bouton rond pour une interface

```
1 import pygame
2 pygame.init()
3
4 GRIS = (200, 200, 200)
5 BLANC = (255, 255, 255)
6 NOIR = (0, 0, 0)
7
8 def bouton(screen, pos, texte):
9     rayon = 75
10    pygame.draw.circle(screen, GRIS, pos, rayon)
11    txtBtn = font.render(texte, True, NOIR)
12    rectBtn = txtBtn.get_rect()
13    rectBtn.center = (pos)
14    screen.blit(txtBtn, rectBtn)
15
16 sysfont = pygame.font.get_default_font()
17 font = pygame.font.SysFont(None, 36)
18
19 screen = pygame.display.set_mode([500, 500])
20
21 running = True
22
23 while running:
24     for event in pygame.event.get():
25         if event.type == pygame.QUIT:
26             running = False
27
28     screen.fill(BLANC)
29
30     bouton(screen, (250, 250), "ON/OFF")
31
32     pygame.display.flip()
33
34 pygame.quit()
```

Affichage des images

Exemple 8 : Charger une image

```
1 import pygame
2 pygame.init()
3
4 BLEU = (0, 0, 255)
5 BLANC = (255, 255, 255)
6
7 screen = pygame.display.set_mode([500, 500])
8
9 # Charger l'image
```

```

10 img = pygame.image.load("bird.png")
11 img.convert()
12
13 running = True
14
15 while running:
16     for event in pygame.event.get():
17         if event.type == pygame.QUIT:
18             running = False
19
20     screen.fill(BLANC)
21
22     screen.blit(img, (250, 250))
23
24     pygame.display.flip()
25
26
27 pygame.quit()

```

Exemple 9 : Centrer une image

```

1  import pygame
2  pygame.init()
3
4  BLEU = (0, 0, 255)
5  BLANC = (255, 255, 255)
6
7  screen = pygame.display.set_mode([500, 500])
8
9  # Charger l'image
10 img = pygame.image.load("bird.png")
11 img.convert()
12 # Centrer l'image
13 rect = img.get_rect()
14 rect.center = 250, 250
15
16 running = True
17
18 while running:
19     for event in pygame.event.get():
20         if event.type == pygame.QUIT:
21             running = False
22
23     screen.fill(BLANC)
24
25     screen.blit(img, rect) # Utiliser rect pour la position
26
27     pygame.display.flip()
28
29
30 pygame.quit()

```

Exemple 10 : Rotation et mise à l'échelle

```
1 import pygame
2 pygame.init()
3
4 BLEU = (0, 0, 255)
5 BLANC = (255, 255, 255)
6
7 screen = pygame.display.set_mode([500, 500])
8
9 # Charger l'image
10 img = pygame.image.load("bird.png")
11 img.convert()
12 # Rotation
13 img = pygame.transform.rotozoom(img, 45, 1) # Angle: 45deg; scale: 1
14 # Centrer l'image
15 rect = img.get_rect()
16 rect.center = 250, 250
17
18 running = True
19
20 while running:
21
22     for event in pygame.event.get():
23         if event.type == pygame.QUIT:
24             running = False
25
26     screen.fill(BLANC)
27
28     screen.blit(img, rect) # Utiliser rect pour la position
29
30     pygame.display.flip()
31
32 pygame.quit()
```

Exemple 11 : Conditions météorologiques (requests)

```
1 import requests
2 import pprint
3
4 api_key = 'xxx';
5 url = 'http://api.openweathermap.org/data/2.5/weather?q=Montreal,ca&
6         lang=fr&units=metric&appid=' + api_key;
7 reponse = requests.get(url)
8 data = reponse.json()
9
10 pp = pprint.PrettyPrinter(indent = 2)
11 pp.pprint(data)
12
```



```

13 texte = u"Conditions actuelles: " + data['weather'][0]['description']
14 texte += u" et il fait " + str(int(round(data['main']['temp'])))
15     + u" degrés celsius"
16 print(texte)
17 print("Le nom de l'icone: " + data['weather'][0]['icon'])

```

Note: Les icônes sont disponibles à l'url: <http://openweathermap.org/img/wn/> + w.weather_icon_name + ".png"

Exemple 12 : Conditions météorologiques (pyowm)

```

1 from pyowm import OWM
2 from pyowm.utils.config import get_default_config
3
4 config_dict = get_default_config()
5 config_dict['language'] = 'fr'
6
7 owm = OWM('xxxx', config_dict)
8 mgr = owm.weather_manager()
9
10 observation = mgr.weather_at_place('Montreal, CA')
11 w = observation.weather
12
13 print("Temperature: " + str(w.temperature('celsius')['temp']))
14 print(w.detailed_status)
15 print("Nom de l'icone: " + w.weather_icon_name)

```

Gestion des évènements

En informatique, un événement peut être une entrée clavier (soit l'appui soit le relâchement d'une touche), le déplacement de votre souris, un clic (encore une fois, appui ou relâchement, qui seront traités comme deux événements distincts). Un bouton de votre joystick peut aussi engendrer un événement, et même la fermeture de votre fenêtre est considéré comme un événement !

De plus, il faut savoir que chaque événement créé est envoyé sur une file (ou queue), en attendant d'être traité. Quand un événement entre dans cette queue, il est placé à la fin de celle-ci. Vous l'aurez donc compris, le premier événement transmis à Pygame sera traité en premier !

Souris

Exemple 13 : Afficher la position du clic de la souris

```
1 import pygame
2 pygame.init()
3
4 BLANC = (255, 255, 255)
5
6 screen = pygame.display.set_mode([500, 500])
7
8 running = True
9
10 while running:
11     for event in pygame.event.get():
12         if event.type == pygame.QUIT:
13             running = False
14         elif event.type == pygame.MOUSEBUTTONDOWN:
15             xSouris, ySouris = pygame.mouse.get_pos()
16             print("Clic souris (" + str(xSouris) + ", " + str(ySouris) + ")")
17
18     screen.fill(BLANC)
19
20     pygame.display.flip()
21
22
23 pygame.quit()
```

Clavier

Exemple 14 : Afficher un message lorsqu'une touche spécifique est pressée

```
1 import pygame
2 pygame.init()
3
4 BLANC = (255, 255, 255)
5
6 screen = pygame.display.set_mode([500, 500])
7
8 running = True
9
10 while running:
11     for event in pygame.event.get():
12         if event.type == pygame.QUIT:
13             running = False
14         elif event.type == pygame.KEYDOWN:
15             if event.key == pygame.K_UP:
16                 print("Fleche vers le haut")
17             elif event.key == pygame.K_a:
```

```

19             print("La touche a")
20
21     screen.fill(BLANC)
22
23     pygame.display.flip()
24
25     pygame.quit()

```

Associer un évènement à un objet graphique

Exemple 15 : Associer un évènement à un bouton

```

1  import pygame
2  import math
3  pygame.init()
4
5  GRIS = (200, 200, 200)
6  BLANC = (255, 255, 255)
7  NOIR = (0, 0, 0)
8
9  def bouton(screen, pos, texte):
10     rayon = 75
11     pygame.draw.circle(screen, GRIS, pos, rayon)
12     txtBtn = font.render(texte, True, NOIR)
13     rectBtn = txtBtn.get_rect()
14     rectBtn.center = (pos)
15     screen.blit(txtBtn, rectBtn)
16
17  def isOverBouton(posBouton, posSouris):
18     rayon = 75
19     xBouton = posBouton[0]
20     yBouton = posBouton[1]
21     xSouris = posSouris[0]
22     ySouris = posSouris[1]
23
24     absX = (xBouton-xSouris)**2
25     absY = (yBouton-ySouris)**2
26
27     return (math.sqrt(absX+absY) < rayon)
28
29
30  sysfont = pygame.font.get_default_font()
31  font = pygame.font.SysFont(None, 36)
32
33  screen = pygame.display.set_mode([500, 500])
34
35  running = True
36
37  while running:
38     for event in pygame.event.get():
39         if event.type == pygame.QUIT:

```

```

40         running = False
41     elif event.type == pygame.MOUSEBUTTONDOWN:
42         xSouris,ySouris = pygame.mouse.get_pos()
43         if isOverBouton((250, 250), (xSouris,ySouris)):
44             print("bouton")
45
46     screen.fill(BLANC)
47
48     bouton(screen, (250, 250), "ON/OFF")
49
50     pygame.display.flip()
51
52     pygame.quit()

```

Exemple 16 : Créer une classe boutons

```

1  import pygame
2  import math
3
4  class objBouton():
5      def __init__(self, couleur, x, y, rayon, texte=''):
6          self.couleur = couleur
7          self.x = x
8          self.y = y
9          self.rayon = rayon
10         self.texte = texte
11
12     def dessiner(self, screen):
13         sysfont = pygame.font.get_default_font()
14         font = pygame.font.SysFont(None, 36)
15         pygame.draw.circle(screen,self.couleur,(self.x, self.y),self.rayon)
16         txtBtn = font.render(self.texte, True, (0, 0, 0))
17         rectBtn = txtBtn.get_rect()
18         rectBtn.center = (self.x, self.y)
19         screen.blit(txtBtn, rectBtn)
20
21     def isOverBouton(self, posSouris):
22         xSouris = posSouris[0]
23         ySouris = posSouris[1]
24
25         absX = (self.x-xSouris)**2
26         absY = (self.y-ySouris)**2
27
28         return (math.sqrt(absX+absY) < self.rayon)
29
30 if __name__ == "__main__":
31     print ("Début du test")
32     pygame.init()
33
34     GRIS = (200, 200, 200)
35     BLANC = (255, 255, 255)
36

```

```

37     screen = pygame.display.set_mode([500, 500])
38
39     btn = objBouton(GRIS, 250, 250, 75, "ON/OFF")
40
41     running = True
42
43     while running:
44         for event in pygame.event.get():
45             if event.type == pygame.QUIT:
46                 running = False
47             elif event.type == pygame.MOUSEBUTTONDOWN:
48                 xSouris,ySouris = pygame.mouse.get_pos()
49                 if btn.isOverBouton((xSouris,ySouris)):
50                     print("Clic bouton!")
51
52         screen.fill(BLANC)
53
54         btn.dessiner(screen)
55
56         pygame.display.flip()
57
58     pygame.quit()
59     print("Fin du test")

```

Détection de collisions

La détection de collisions peut-être effectué en utilisant la classe `pygame.Rect`. Cette classe fournit plusieurs méthodes de détection de collision entre différents objets.

Exemple 17 : Vérifier si un point est à l'intérieur d'un rectangle

```

1  import pygame
2
3  ROUGE = (255, 0, 0)
4  BLANC = (255, 255, 255)
5  NOIR = (0, 0, 0)
6
7  pygame.init()
8  screen = pygame.display.set_mode((250, 250))
9  rect = pygame.Rect(*screen.get_rect().center, 0, 0).inflate(100, 100)
10
11  run = True
12  while run:
13      for event in pygame.event.get():
14          if event.type == pygame.QUIT:
15              run = False
16
17      point = pygame.mouse.get_pos()
18

```

```

19     if rect.collidepoint(point):
20         couleur = BLANC
21     else:
22         couleur = ROUGE
23
24     screen.fill(NOIR)
25     pygame.draw.rect(screen, couleur, rect)
26     pygame.display.flip()
27
28 pygame.quit()

```

pygame.Rect est un objet qui contient les coordonnées d'un rectangle.

Exemple 18 : Vérifier si un rectangle est à l'intérieur d'un rectangle

```

1  import pygame
2
3  ROUGE = (255, 0, 0)
4  BLANC = (255, 255, 255)
5  NOIR = (0, 0, 0)
6  VERT = (0, 255, 0)
7
8  pygame.init()
9  screen = pygame.display.set_mode((250, 250))
10 rect1 = pygame.Rect(*screen.get_rect().center, 0, 0).inflate(75, 75)
11 rect2 = pygame.Rect(0, 0, 75, 75)
12
13 run = True
14 while run:
15     for event in pygame.event.get():
16         if event.type == pygame.QUIT:
17             run = False
18
19     rect2.center = pygame.mouse.get_pos()
20
21     if rect1.colliderect(rect2):
22         couleur = BLANC
23     else:
24         couleur = ROUGE
25
26     screen.fill(NOIR)
27     pygame.draw.rect(screen, couleur, rect1)
28     pygame.draw.rect(screen, VERT, rect2, 6, 1)
29     pygame.display.flip()
30
31 pygame.quit()

```

Exemple 19 : Vérifier si un rectangle est à l'intérieur d'un rectangle en utilisant les « Sprites »

```

1  import pygame

```

```

2
3 ROUGE = (255, 0, 0)
4 BLANC = (255, 255, 255)
5 NOIR = (0, 0, 0)
6 VERT = (0, 255, 0)
7 BLEU = (0, 0, 255)
8
9 class Brique(pygame.sprite.Sprite):
10     def __init__(self, color, x, y, width, height):
11         pygame.sprite.Sprite.__init__(self)
12
13         self.image = pygame.Surface([width, height])
14         self.image.fill(color)
15         self.rect = self.image.get_rect()
16         self.rect.x = x
17         self.rect.y = y
18
19 pygame.init()
20 screen = pygame.display.set_mode((400, 400))
21
22 brique1 = Brique(ROUGE, 0, 0, 50, 50)
23 brique2 = Brique(VERT, 100, 100, 50, 50)
24 brique3 = Brique(BLEU, 250, 250, 30, 30)
25
26 all_group = pygame.sprite.Group([brique1, brique2, brique3])
27 test_group = pygame.sprite.Group([brique2, brique3])
28
29 run = True
30 while run:
31     for event in pygame.event.get():
32         if event.type == pygame.QUIT:
33             run = False
34
35     brique1.rect.center = pygame.mouse.get_pos()
36     collide = pygame.sprite.spritecollide(brique1, test_group, False)
37
38     screen.fill(NOIR)
39     all_group.draw(screen)
40
41     for s in collide:
42         pygame.draw.rect(screen, BLANC, s.rect, 5, 1)
43     pygame.display.flip()
44
45 pygame.quit()

```

Exemple 20 : Créer un dialogue modal

```

1 import pygame
2
3 ROUGE = (255, 0, 0)
4 BLANC = (255, 255, 255)
5 NOIR = (0, 0, 0)

```

```

6  VERT = (0, 255, 0)
7  BLEU = (0, 0, 255)
8
9  class Bouton():
10     def __init__(self, color, x, y, width, height, texte=''):
11
12         self.taillePolice = 24
13         self.couleurPolice = NOIR
14         self.texte = texte
15
16         sysfont = pygame.font.get_default_font()
17         font = pygame.font.SysFont(None, self.taillePolice)
18         text = font.render(self.texte, True, self.couleurPolice)
19         (wt, ht) = text.get_size()
20         self.image = pygame.Surface([width, height])
21         self.image.fill(color)
22         self.image.blit(text, ((width-wt)/2, (height-ht)/2))
23         self.rect = self.image.get_rect()
24         self.rect.x = x
25         self.rect.y = y
26
27     def afficher(self, screen):
28         screen.blit(self.image, (self.rect.x, self.rect.y))
29
30 class Menu():
31     def __init__(self, texte):
32         self.texte = texte
33         self.taillePolice = 24
34         self.couleurPolice = NOIR
35
36         self.bouton1 = Bouton(VERT, 20, 60, 100, 40, "OUI")
37         self.bouton2 = Bouton(ROUGE, 20, 120, 100, 40, "NON")
38
39         self.boutons = [self.bouton1, self.bouton2]
40
41     def afficher(self, screen):
42         sysfont = pygame.font.get_default_font()
43         font = pygame.font.SysFont(None, self.taillePolice)
44         txtBtn = font.render(self.texte, True, self.couleurPolice)
45         screen.blit(txtBtn, (20, 20))
46
47         for bouton in self.boutons:
48             bouton.afficher(screen)
49
50     def obtenir_choix(self, point):
51         for index, bouton in enumerate(self.boutons):
52             if bouton.rect.collidepoint(point) == True:
53                 return index
54         return -1
55
56 def test():
57     print ("Début du test")
58     pygame.init()
59
60     menu1 = Menu("Ceci est une question?")

```



```

61
62     screen = pygame.display.set_mode([500, 500])
63
64     running = True
65
66     while running:
67         for event in pygame.event.get():
68             if event.type == pygame.QUIT:
69                 running = False
70             elif event.type == pygame.MOUSEBUTTONDOWN:
71                 point = pygame.mouse.get_pos()
72                 print(menu1.obtenir_choix(point))
73
74         screen.fill(BLANC)
75
76         menu1.afficher(screen)
77
78         pygame.display.flip()
79
80     pygame.quit()
81     print("Fin du test")
82
83 if __name__ == "__main__":
84     test()

```

Exemple 21 : Gérer des dialogues modaux

```

1  from enum import Enum
2  from E04_dialog import Menu
3  import pygame
4
5  class Menu_etat(Enum):
6      MENU1 = 0
7      MENU2 = 1
8
9  BLANC = (255, 255, 255)
10 GRIS = (100, 100, 100)
11
12 pygame.init()
13
14 menu1 = Menu("Menu 1")
15 menu2 = Menu("Menu 2")
16
17 screen = pygame.display.set_mode([500, 500])
18
19 running = True
20 etat = Menu_etat.MENU1
21
22 while running:
23     for event in pygame.event.get():
24         if event.type == pygame.QUIT:
25             running = False

```

```

26         elif event.type == pygame.MOUSEBUTTONDOWN:
27             point = pygame.mouse.get_pos()
28             if etat == Menu_etat.MENU1:
29                 if menu1.obtenir_choix(point) == 0:
30                     etat = Menu_etat.MENU2
31             elif etat == Menu_etat.MENU2:
32                 if menu2.obtenir_choix(point) == 0:
33                     etat = Menu_etat.MENU1
34
35         if etat == Menu_etat.MENU1:
36             screen.fill(BLANC)
37             menu1.afficher(screen)
38         elif etat == Menu_etat.MENU2:
39             screen.fill(GRIS)
40             menu2.afficher(screen)
41
42         pygame.display.flip()
43
44     pygame.quit()

```

Musique

```

1  import pygame
2  file = 'assets/Area-2Music.ogg'
3  pygame.init()
4  pygame.mixer.init()
5  pygame.mixer.music.load(file)
6  pygame.mixer.music.play(-1)
7
8  running = True
9
10 while running:
11
12     for event in pygame.event.get():
13         if event.type == pygame.QUIT:
14             running = False
15
16 pygame.quit()

```

À la ligne 6, la valeur -1 est assigné au paramètre loop de la méthode play, ce qui permet de lire le fichier de musique en boucle.

Manipulation d'image

Qu'est-ce que Pillow? Pillow (Python Imaging Library - PIL) est une bibliothèque de traitement d'images pour le langage de programmation Python. Elle permet d'ouvrir, de manipuler, et de sauvegarder différents formats de fichiers d'images.

Exemple 22 : Charger un fichier d'image

```
1 from PIL import Image
2
3 img = Image.open("Paris.jpg")
4 print(img.format, img.size, img.mode)
```

Exemple 23 : Extraire l'information EXIF

```
1 from PIL import Image, ExifTags
2 img = Image.open("Paris.jpg")
3
4 for orientation in ExifTags.TAGS.keys():
5     if ExifTags.TAGS[orientation]=='Orientation':
6         break
7
8 print("Orientation (Indice) : ", orientation)
9 try:
10     exifData = img._getexif()
11     print("Orientation (Valeur) : ", exifData[orientation])
12 except:
13     print("Pas de donnée EXIF")
```

Exemple 24 : Convertir en JPEG

```
1 import os
2 from PIL import Image
3
4 fichierImg = "bird.png"
5
6 nomFichier, extensionFichier = os.path.splitext(fichierImg)
7 print(nomFichier, extensionFichier)
8
9 try:
10     img = Image.open(fichierImg)
11     rgbImg = img.convert('RGB')
12     rgbImg.save(nomFichier + ".jpg")
13 except OSError:
14     print("Impossible de convertir", fichierImg)
```

Exemple 25 : Générer une vignette

```
1 import os
2 from PIL import Image
3
4 fichierImg = "bird.png"
5 taille = (128, 128)
6
7 nomFichier, extensionFichier = os.path.splitext(fichierImg)
8 print(nomFichier, extensionFichier)
9
10 try:
11     img = Image.open(fichierImg)
12     rgbImg = img.convert('RGB')
13     rgbImg.thumbnail(taille)
14     rgbImg.save(nomFichier + ".thumbnail", "JPEG")
15 except OSError:
16     print("Impossible de convertir", fichierImg)
```

Exemple 26 : Appliquer des transformations à une image

```
1 import os
2 from PIL import Image
3
4 fichierImg = "bird.png"
5
6 nomFichier, extensionFichier = os.path.splitext(fichierImg)
7 print(nomFichier, extensionFichier)
8
9 try:
10     img = Image.open(fichierImg)
11     rgbImg = img.convert('RGB')
12     rgbImg = rgbImg.resize((128, 128))
13     rgbImg = rgbImg.rotate(45)
14     rgbImg.save(nomFichier + ".jpg", "JPEG")
15 except OSError:
16     print("Impossible de convertir", fichierImg)
```

Références

- https://fr.wikibooks.org/wiki/Pygame/Introduction_%C3%A0_Pygame
- https://github.com/Rabbid76/PyGameExamplesAndAnswers/blob/master/documentation/pygame/pygame_collision_and_intesection.md
- https://www.reddit.com/r/pygame/comments/hblj83/circular_button_with_variable_width_and_height/

- Vidéo du cours 8 : <https://youtu.be/sfQvDFV2RtI>
- Vidéo du cours 9 : <https://youtu.be/1ER2S7FStBQ>
- Vidéo du cours 10 : <https://youtu.be/-umjTizNXA4>
- Vidéo du cours 11 : <https://youtu.be/4Sp8L4IMTrQ>
- Vidéo du cours 15 (PIL) : <https://youtu.be/dbWuwPizPjs>